10. A* ALGORITHM

Aim:

To implement the A* (A-Star) Search Algorithm in Python to find the shortest path between a source node and a goal node using heuristic information.

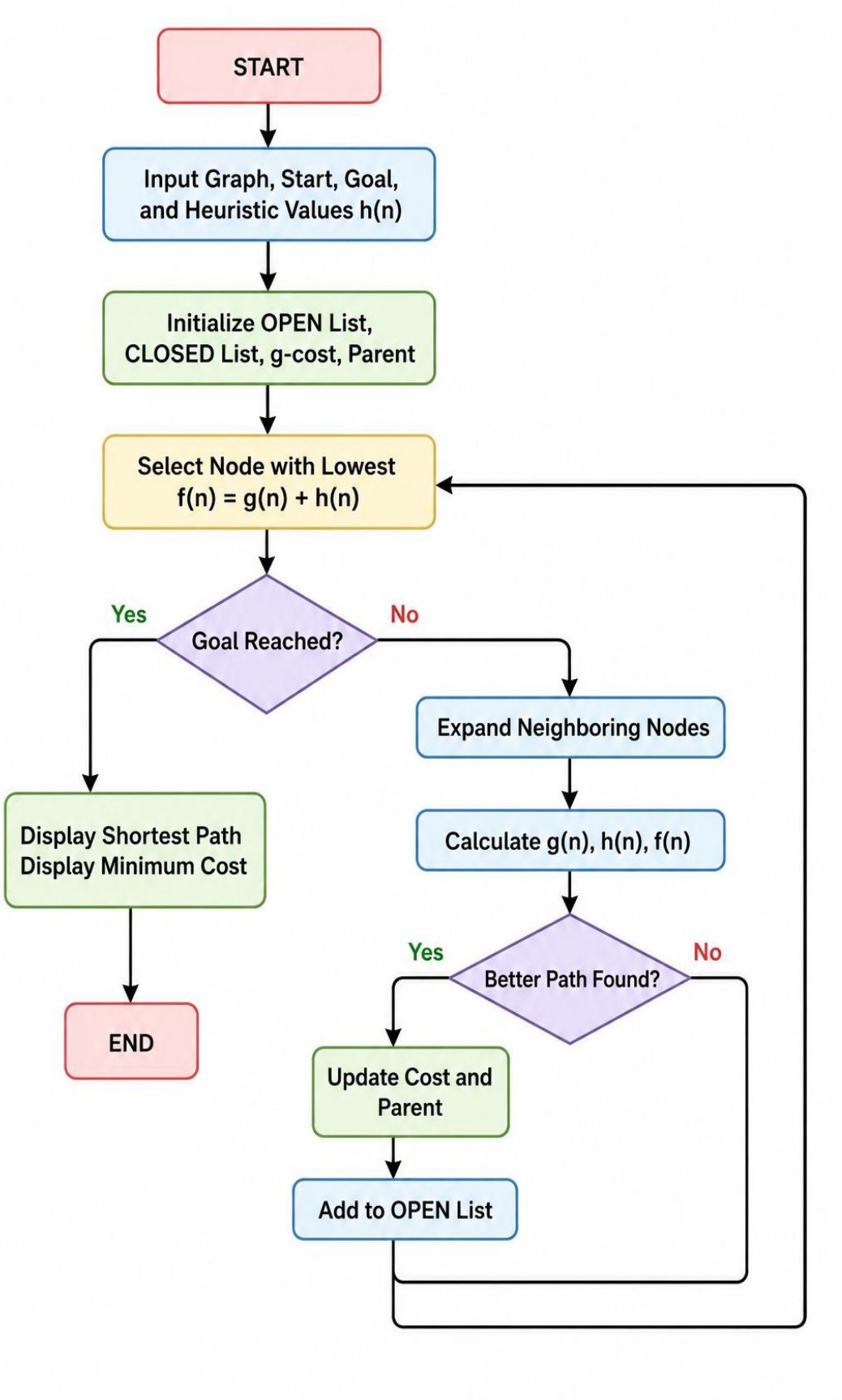
Objective :

1. To understand the working of the A* Search Algorithm.
2. To find the optimal path from the start node to the goal node.
3. To minimize the total path cost using both the actual cost and heuristic value.
4. To demonstrate informed search in Artificial Intelligence.

Algorithm:

1. Create a graph with nodes and edge costs.
2. Define heuristic values for each node.
3. Initialize the OPEN list (priority queue) with the start node.
4. Initialize the CLOSED set as empty.
5. Remove the node with the lowest **f(n) = g(n) + h(n)** from the OPEN list.
6. If the current node is the goal node, reconstruct and display the shortest path.
7. Otherwise, expand all neighboring nodes.
8. Compute:
    - **g(n)** = Actual path cost from the start node.
    - **h(n)** = Heuristic estimate to the goal.
    - **f(n)** = g(n) + h(n).
9. If a better path is found, update the node's cost and parent.
10. Repeat until the goal node is reached or the OPEN list becomes empty.

Flowchart :

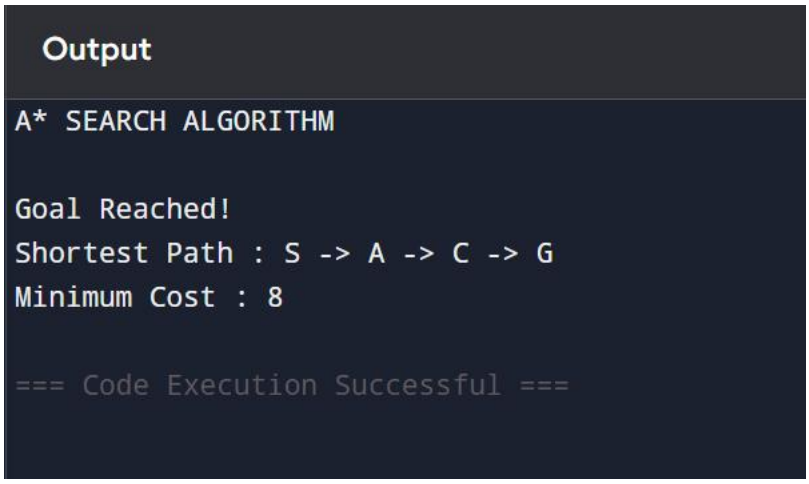

START
Input Graph, Start, Goal, and Heuristic Values h(n)
Initialize OPEN List, CLOSED List, g-cost, Parent
Select Node with Lowest f(n) = g(n) + h(n)
Goal Reached?
Yes
No
Display Shortest Path
Display Minimum Cost
END
Expand Neighboring Nodes
Calculate g(n), h(n), f(n)
Better Path Found?
Yes
No
Update Cost and Parent
Add to OPEN List

Code:

```
import heapq
graph = {
    'S': [('A', 1), ('B', 4)],
    'A': [('C', 2), ('D', 5)],
    'B': [('D', 1)],
    'C': [('G', 5)],
    'D': [('G', 3)],
    'G': []
}
heuristic = {
    'S': 7,
    'A': 6,
    'B': 4,
    'C': 2,
    'D': 1,
    'G': 0
}
def a_star(start, goal):
    open_list = []
    heapq.heappush(open_list, (heuristic[start], start))
    g_cost = {start: 0}
    parent = {start: None}
    closed = set()
    while open_list:
```

```
        current_f, current = heapq.heappop(open_list)
        if current == goal:
            path = []
            while current is not None:
                path.append(current)
                current = parent[current]
            path.reverse()
            print("\nGoal Reached!")
            print("Shortest Path :", " -> ".join(path))
            print("Minimum Cost :", g_cost[goal])
            return
        if current in closed:
            continue
        closed.add(current)
        for neighbor, cost in graph[current]:
            new_g = g_cost[current] + cost
            if neighbor not in g_cost or new_g < g_cost[neighbor]:
                g_cost[neighbor] = new_g
                parent[neighbor] = current
                f_cost = new_g + heuristic[neighbor]
                heapq.heappush(open_list, (f_cost, neighbor))
    print("No Path Exists.")
print("A* SEARCH ALGORITHM")
start_node = 'S'
goal_node = 'G'
a_star(start_node, goal_node)
```

Output:



Result :

The A* Search Algorithm was successfully implemented using Python. The algorithm found the optimal path from the start node (**S**) to the goal node (**G**) by considering both the actual path cost and heuristic values.